(Instituto Federal Campos do Jordão)
(Analise dados e desenvolvimento de sistemas)

(Lucas Augusto Oliveira Silva)

CÁLCULO RELACIONAL

Fundamentos, aplicações e exemplos de consultas

Trabalho de pesquisa apresentado à disciplina de Banco de Dados.

(Paulo Giovani de Faria Zeferino)

(Campos do Jordão)

2026

1. Introdução

O **cálculo relacional** é uma das linguagens formais de consulta associadas ao modelo relacional de dados, proposto por Edgar Frank Codd na década de 1970 enquanto pesquisador da IBM. Trata-se de uma linguagem de consulta **declarativa** (também chamada de não procedural): em vez de descrever *como* os dados devem ser obtidos, o usuário especifica apenas *o que* deseja recuperar do banco de dados. A tarefa de descobrir a sequência de operações necessária para produzir esse resultado fica a cargo do Sistema Gerenciador de Banco de Dados (SGBD).

Essa característica diferencia o cálculo relacional da **álgebra relacional**, a outra linguagem formal do modelo relacional. A álgebra é *procedural*: descreve o resultado por meio de uma sequência de operações (seleção, projeção, junção, união, etc.) aplicadas sobre as relações. Já o cálculo descreve o resultado por meio de uma *expressão lógica* que caracteriza os dados desejados. Apesar dessa diferença de estilo, as duas linguagens são equivalentes em poder de expressão, como será detalhado adiante.

O cálculo relacional é fundamentado na **lógica de predicados de primeira ordem**. Ele possui duas variantes equivalentes, que se distinguem apenas pelo tipo de variável utilizada:

- **Cálculo Relacional de Tuplas (CRT):** as variáveis representam tuplas (linhas) inteiras de uma relação.
- **Cálculo Relacional de Domínio (CRD):** as variáveis representam valores de domínio, ou seja, valores individuais de atributos (colunas).

Ambas expressam exatamente o mesmo conjunto de consultas; a escolha entre uma e outra é, em geral, uma questão de conveniência ou de clareza. Este trabalho apresenta os fundamentos teóricos do cálculo relacional, discute onde ele é utilizado e por que seu estudo é importante, e ilustra o assunto com exemplos práticos de consultas nas duas notações, acompanhados de seus equivalentes em SQL.

2. Fundamentos do Cálculo Relacional

2.1 Base na lógica de primeira ordem

Uma consulta em cálculo relacional é escrita como uma **fórmula,** uma expressão da lógica de primeira ordem que pode ser avaliada como verdadeira ou falsa para cada combinação de valores. O resultado da consulta é o conjunto de todas as atribuições de valores às variáveis livres que tornam a fórmula verdadeira. Os elementos disponíveis para construir fórmulas são:

- **Variáveis** (de tupla ou de domínio) e **constantes** (por exemplo, 'Sistemas de Informação' ou o número 19).
- **Operadores de comparação:** =, ≠, <, >, ≤ e ≥.
- **Conectivos lógicos:** ∧ (E / conjunção), ∨ (OU / disjunção), ¬ (NÃO / negação) e → (implicação).
- **Quantificadores:** ∃ (existencial, "existe") e ∀ (universal, "para todo").

2.2 Cálculo Relacional de Tuplas (CRT)

No CRT, uma consulta tem a forma geral:

```
{ t | P(t) }
```

onde **t** é uma variável de tupla e **P(t)** é uma fórmula (predicado) que descreve as condições que a tupla t deve satisfazer. O resultado é o conjunto de todas as tuplas t para as quais P(t) é verdadeira. Quando se deseja recuperar apenas alguns atributos, escreve-se a forma estendida, indicando os atributos desejados antes da barra vertical:

```
{ t.A1, t.A2, ..., t.An | P(t) }
```

A notação **t.matricula**, por exemplo, denota o valor do atributo *matricula* na tupla t. A fórmula atômica **R(t)** é verdadeira quando a tupla t pertence à relação R; além dela, as fórmulas atômicas podem ser comparações entre atributos de tuplas (t.A = s.B) ou entre um atributo e uma constante (t.A = c).

2.3 Cálculo Relacional de Domínio (CRD)

No CRD, as variáveis representam valores de domínio (valores de colunas). A forma geral é:

```
{ x1, x2, ..., xn | P(x1, x2, ..., xn) }
```

onde cada **xi** é uma variável de domínio. A fórmula atômica **R(x1, x2, ..., xn)** é verdadeira quando existe, na relação R, uma tupla cujos valores de atributos são exatamente x1, x2, ..., xn, na ordem das colunas. A linguagem visual de consulta **QBE** (*Query-By-Example*) é baseada no cálculo relacional de domínio.

2.4 Quantificadores: existencial e universal

Os quantificadores são o que dá ao cálculo relacional grande parte de seu poder expressivo. A expressão **∃t (P(t))** lê-se "existe pelo menos uma tupla t que satisfaz P"; ela é verdadeira se houver ao menos um valor de t que torne P verdadeira. Já **∀t (P(t))** lê-se "para toda tupla t, P é verdadeira"; ela só é verdadeira se P valer para todos os valores possíveis de t. Uma variável ligada por um quantificador é chamada *variável ligada*; as demais, que aparecem antes da barra e determinam o resultado, são as *variáveis livres*.

2.5 Expressões seguras

Nem toda fórmula bem formada produz um resultado útil. Considere a expressão { t | ¬ALUNO(t) }, que pediria "todas as tuplas que não estão na relação ALUNO". Esse conjunto é **infinito**, pois inclui qualquer combinação de valores fora da relação. Para evitar esse problema, restringe-se o cálculo às chamadas **expressões seguras** (*safe expressions*), cujo resultado é garantidamente finito é formado apenas por valores presentes no banco de dados. Na prática, toda consulta útil pode ser escrita como uma expressão segura.

2.6 Completude relacional (Teorema de Codd)

Em seu artigo de 1972, *Relational Completeness of Data Base Sublanguages*, Codd definiu formalmente o cálculo relacional e apresentou um algoritmo capaz de transformar qualquer expressão do cálculo em uma expressão equivalente da álgebra relacional. Com isso, demonstrou o resultado hoje conhecido como **Teorema de Codd**: a álgebra relacional e o cálculo relacional (restrito às expressões seguras) têm exatamente o **mesmo poder de expressão,** uma consulta pode ser formulada em uma das linguagens se, e somente se, puder ser formulada na outra. Uma linguagem de consulta com poder de expressão igual ou superior ao da álgebra relacional é chamada de **relacionalmente completa**. Esse conceito tornou-se um critério de referência para avaliar a expressividade de linguagens de consulta: a SQL, por exemplo, é relacionalmente completa e ainda acrescenta recursos que vão além do cálculo, como funções de agregação (contagem, soma, média, etc.).

3. Onde o Cálculo Relacional é Utilizado

Embora o cálculo relacional seja, em si, uma linguagem teórica, programadores não escrevem cálculo relacional diretamente no dia a dia, ele está na base de diversas tecnologias e ideias amplamente usadas na prática:

- **Fundamento das linguagens de consulta declarativas (SQL):** a própria natureza declarativa da SQL, em que se descreve o resultado desejado e o SGBD decide como obtê-lo, deriva do cálculo relacional. A linguagem ALPHA, proposta por Codd em 1971 e baseada no cálculo relacional, influenciou a linguagem QUEL (do SGBD Ingres) e, de forma mais indireta, a SQL. Construções da SQL como EXISTS e NOT EXISTS correspondem diretamente aos quantificadores ∃ e ¬∃ do cálculo.
- **Query-By-Example (QBE):** linguagem de consulta visual, na qual o usuário monta consultas e preenchimento de tabelas-exemplo, é fundamentada no cálculo relacional de domínio.
- **Otimização de consultas:** como o cálculo (e a SQL) apenas descrevem o resultado, o otimizador do SGBD tem liberdade para escolher o plano de execução mais eficiente. A equivalência entre cálculo e álgebra, garantida pelo
  Teorema de Codd, é justamente o que permite traduzir uma consulta declarativa em um plano algébrico que pode então ser otimizado.
- **Bancos de dados dedutivos e Datalog:** linguagens dedutivas como o Datalog compartilham a mesma base lógica (lógica de primeira ordem) do cálculo relacional, estendendo-a com recursão e regras.
- **Ensino e fundamentação teórica:** o cálculo relacional é parte padrão dos cursos de banco de dados porque explicita o significado lógico das consultas e formaliza a noção de completude relacional, servindo de semântica formal para linguagens práticas.
- **Raciocínio formal sobre consultas:** por ser baseado em lógica, o cálculo permite provar a equivalência entre consultas e raciocinar formalmente sobre sua correção.

4. Por que seu Estudo é Importante

O estudo do cálculo relacional tem valor tanto teórico quanto prático para quem trabalha com dados e desenvolvimento de software:

- **Compreender a origem da SQL:** entender o cálculo ajuda a compreender *por que* a SQL funciona, como funciona e qual é o significado preciso de cada consulta, indo além da simples memorização de comandos.
- **Desenvolver o pensamento declarativo:** o cálculo treina o raciocínio focado no *resultado desejado*, e não no passo a passo, exatamente a mentalidade exigida pela SQL. Isso tende a produzir consultas mais corretas e bem estruturadas.
- **Dominar a noção de completude relacional:** ela fornece um critério objetivo para avaliar e comparar o poder de expressão de diferentes linguagens de consulta.
- **Conectar lógica e manipulação de dados:** os quantificadores $\exists$ e $\forall$ correspondem a padrões reais de consulta (existência de registros relacionados e a operação de "divisão", em que se busca algo relacionado a *todos* os itens de um conjunto).
- **Servir de base para tópicos avançados:** otimização de consultas, bancos de dados dedutivos e a semântica formal de linguagens de consulta apoiam-se nesses fundamentos.

5. Exemplos de Consultas

Para ilustrar o uso do cálculo relacional, considere um banco de dados simples de uma instituição de ensino, formado por três relações. As chaves primárias estão sublinhadas na descrição a seguir: **ALUNO**(matricula, nome, curso, idade),
DISCIPLINA(código, título, créditos) e **MATRÍCULA**(matrícula, código, nota), em que *matricula* e *código*, na relação MATRÍCULA, referenciam respectivamente ALUNO e DISCIPLINA. As tabelas abaixo mostram um pequeno conjunto de dados de exemplo.

matricula	nome	curso	idade
1	Ana Souza	Sistemas de Informação	20
2	Bruno Lima	Ciência de Dados	22
3	Carla Dias	Sistemas de Informação	19

codigo	titulo	creditos
BD01	Banco de Dados I	4
PR01	Programação I	4
MD01	Matemática Discreta	4

matricula	codigo	nota
1	BD01	8.5
1	PR01	7.0
2	BD01	9.0
3	BD01	6.5
3	PR01	8.0
3	MD01	7.5

Consulta 1 — Seleção simples

Recuperar o nome de todos os alunos do curso de Sistemas de Informação.

CRT: { t.nome | ALUNO(t) ∧ t.curso = 'Sistemas de Informação' } **CRD:** { n | ∃m ∃c ∃i (ALUNO(m, n, c, i) ∧ c = 'Sistemas de Informação') } **SQL:** SELECT nome FROM ALUNO WHERE curso = 'Sistemas de Informação';

No CRT, a fórmula pede as tuplas t da relação ALUNO cujo atributo *curso* é igual à constante indicada, retornando apenas o atributo *nome*. No CRD, as variáveis de domínio m, n, c e i representam, respectivamente, matrícula, nome, curso e idade; o quantificador existencial afirma que existe uma tupla em ALUNO com esses valores e com o curso desejado. Resultado: Ana Souza e Carla Dias.

Consulta 2 — Projeção de vários atributos com condição

Recuperar a matrícula e o nome dos alunos com mais de 19 anos.

CRT: { t.matricula, t.nome | ALUNO(t) ∧ t.idade > 19 }
CRD: { m, n | ∃c ∃i (ALUNO(m, n, c, i) ∧ i > 19) }
SQL: SELECT matricula, nome FROM ALUNO WHERE idade > 19;
Aqui são projetados dois atributos. A condição de comparação (idade maior que 19) filtra as tuplas. Resultado: (1, Ana Souza) e (2, Bruno Lima).

Consulta 3 — Quantificador existencial (registros relacionados)

Recuperar o nome dos alunos que estão matriculados em pelo menos uma disciplina.

CRT: { a.nome | ALUNO(a) ∧ ∃m (MATRICULA(m) ∧ m.matricula = a.matricula) } **SQL:** SELECT DISTINCT a.nome

```
FROM ALUNO a
WHERE EXISTS ( SELECT * FROM MATRICULA m
                    WHERE m.matricula = a.matricula );
```

O quantificador existencial ∃ exige apenas que exista *alguma* tupla em MATRICULA associada ao aluno. Note a correspondência direta entre ∃ no cálculo e a cláusula EXISTS da SQL. Como todos os alunos do exemplo possuem matrículas, o resultado inclui Ana, Bruno e Carla.

Consulta 4 — Junção de relações

Recuperar o nome do aluno e o título das disciplinas em que ele está matriculado.

CRT: { a.nome, d.titulo | ALUNO(a) ∧ DISCIPLINA(d) ∧
∃m (MATRICULA(m) ∧ m.matricula = a.matricula ∧ m.codigo = d.codigo) } **SQL:** SELECT a.nome, d.titulo

```
FROM ALUNO a, DISCIPLINA d, MATRICULA m
WHERE m.matricula = a.matricula AND m.codigo = d.codigo;
```

Esta consulta combina (junta) três relações. As igualdades entre atributos (m.matricula = a.matricula e m.codigo = d.codigo) desempenham o papel das condições de junção, ligando cada matrícula ao aluno e à disciplina correspondentes.

Consulta 5 — Quantificador universal (operação de divisão) *Recuperar o nome dos alunos que estão matriculados em todas as disciplinas.*

CRT: { a.nome | ALUNO(a) ∧ ∀d (DISCIPLINA(d) →
∃m (MATRICULA(m) ∧ m.matricula = a.matricula ∧ m.codigo = d.codigo)) }

Esta é a consulta mais expressiva do conjunto e mostra a elegância do quantificador universal. A fórmula lê-se: "para toda disciplina d, existe uma matrícula m que liga o aluno a a essa disciplina". Em outras palavras, não pode haver nenhuma disciplina em que o aluno não esteja matriculado. No conjunto de exemplo, apenas **Carla Dias** satisfaz a condição, pois está matriculada em BD01, PR01 e MD01 (todas as disciplinas). Esse padrão corresponde à operação de *divisão* da álgebra relacional; na SQL, ele costuma ser expresso com uma dupla negação (NOT EXISTS ... NOT EXISTS ...), notoriamente menos intuitiva o que evidencia como o cálculo relacional expressa esse tipo de consulta de forma mais natural e direta.

Consulta 6 — Negação

Recuperar o nome dos alunos que não estão matriculados na disciplina 'BD01'.

CRT: { a.nome | ALUNO(a) ∧ ¬∃m (MATRICULA(m) ∧
m.matricula = a.matricula ∧ m.codigo = 'BD01') }

```
SQL: SELECT a.nome FROM ALUNO a
     WHERE NOT EXISTS ( SELECT * FROM MATRICULA m
                        WHERE m.matricula = a.matricula AND m.codigo = 'BD01' );
```

A negação ¬ combinada com o existencial expressa a ausência de um registro relacionado. É importante que a fórmula esteja "ancorada" em ALUNO(a): isso a torna uma expressão segura, com resultado finito. No exemplo, como Ana, Bruno e Carla estão todos matriculados em BD01, o resultado é vazio.

6. Conclusão

O cálculo relacional é uma linguagem formal e declarativa de consulta, fundamentada na lógica de predicados de primeira ordem, que descreve *o que* se deseja recuperar de um banco de dados sem especificar *como* obtê-lo. Apresenta-se em duas variantes equivalentes o cálculo de tuplas e o cálculo de domínio e, pelo Teorema de Codd, possui o mesmo poder de expressão da álgebra relacional, constituindo com ela a base teórica do modelo relacional.

Ainda que não seja usado diretamente na programação cotidiana, o cálculo relacional é o alicerce conceitual das linguagens de consulta modernas: explica a semântica da SQL, fundamenta linguagens visuais como o QBE, sustenta a otimização de consultas e conecta a lógica formal à manipulação de dados. Compreendê-lo significa entender *por que* as consultas funcionam, desenvolver o raciocínio declarativo e dominar a noção de completude relacional competências valiosas para qualquer pessoa que trabalhe com bancos de dados, análise de dados e desenvolvimento de software. Por isso, embora escrevemos SQL no dia a dia, é o cálculo relacional que dá sentido e rigor a essa prática.

Referências

CODD, E. F. A relational model of data for large shared data banks. *Communications of the ACM*, New York, v. 13, n. 6, p. 377-387, jun. 1970.

CODD, E. F. Relational completeness of data base sublanguages. In: RUSTIN, R. (ed.). *Data Base Systems*: Courant Computer Science Symposia Series 6. Englewood Cliffs: Prentice-Hall, 1972. p. 65-98.

DATE, C. J. *Introdução a sistemas de bancos de dados*. Tradução da 8. ed. Rio de Janeiro: Elsevier, 2004.

ELMASRI, R.; NAVATHE, S. B. *Sistemas de banco de dados*. 6. ed. São Paulo: Pearson Addison Wesley, 2011.

RAMAKRISHNAN, R.; GEHRKE, J. *Database management systems*. 3rd ed. New York: McGraw-Hill, 2003.

SILBERSCHATZ, A.; KORTH, H. F.; SUDARSHAN, S. *Sistema de banco de dados*. Rio de Janeiro: Elsevier, 2006.

CÁLCULO relacional. Notas de aula. Instituto de Matemática e Estatística, Universidade de São Paulo (IME-USP). Disponível em: https://www.ime.usp.br/~jef/calcrelac.pdf. Acesso em: 10 set. 2026.